



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Aldo Hernández Fontes

Asignatura: Modelos de Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 4

Integrante(s): Jacques Sojo Leonardo

Gonzaga Vargas Daniel Wilfrido

Jimenez Arrieta Josue

No. de lista o brigada: 5 y 3

Semestre: 2027-1

Fecha de entrega: Domingo 27 de septiembre

Observaciones:

CALIFICACIÓN: _____

Introducción:

Los tipos de datos hacen referencia al tipo de información que se trabaja, donde la unidad mínima de almacenamiento es el dato, también se puede considerar como el rango de valores que puede tomar una variable durante la ejecución del programa. Un tipo de datos define un conjunto de valores y las operaciones sobre estos valores.

Casi todos los lenguajes de programación explícitamente incluyen la notación del tipo de datos, aunque lenguajes diferentes pueden usar terminologías diferentes. La mayor parte de los lenguajes de programación permiten al programador definir tipos de datos adicionales, normalmente combinando múltiples elementos de otros tipos y definiendo las operaciones del nuevo tipo de dato.

Los valores que pueden adquirir los tipos de datos se manipulan durante la ejecución de un programa a través de variables o arreglos.

Objetivo de la práctica:

Utilizar variables y arreglos con tipos de datos primitivos y con bibliotecas propias del lenguaje.

Descripción del problema 1:

Leer 10 números y mostrar el mayor y el menor.

Diseño de la solución:

Requerimientos:

El problema solicita que leamos 10 números y los ordenemos de mayor a menor con las siguientes especificaciones:

- Nos pide mostrar diez números del mayor al menor sin especificar cómo hacerlo
- No se especifica que tipos de estructuras de repetición se deben usar
- No especifica tipo de valores a usar (float, int, double o los tres)
- No especifica rango de los números (negativos, positivos o ambos)
- No especifica la forma en que se van a leer los datos (Scanner o terminal)
- No se especifica si se debe usar más de una clase o no

Solución:

Debido a que el ejercicio no tiene restricciones se decidió utilizar la biblioteca Scanner para obtener los 10 valores del usuario, y utilizar 2 ciclos for y un ciclo anidado basado en el ciclo de la burbuja. Se utilizaron los ciclos for porque se consideró que eran las opciones más eficientes para el ejercicio.

Implementación:

La parte más relevante del código sería el “for anidado” que sirve para ordenar los valores por medio del ciclo de la burbuja, dentro del segundo “for” se puso una condición “if” que permitía ver si la posición “n[v]” del arreglo era menor a la siguiente (n[v+1]) para así intercambiarlas.

Para poder intercambiar posiciones era necesario utilizar una variable auxiliar (z) que guardara la variable de la posición “n[v]” para posteriormente asignarle el valor de “n[v+1]” a “n[v]” y al final a “n[v+1]” darle el valor de “z”.

```
for(v=0;v<9;v++){ //Se pon
    for (y=0; y<9 ;y++) {
        if(n[y]<n[y+1]) {
            z=n[y];
            n[y]=n[y+1];
            n[y+1]=z;
        }
    }
}
//Mismo caso para este for
```

Observaciones:

En sí el código no fue difícil pero se tuvo una que otra complicación para hacer el for anidado debido a que no se tenía bien definida la idea de cómo se iba a acomodar el arreglo de mayor a menor.

Pruebas y resultados:

Números ingresados de menor a mayor

```
terminador para Application) en program resources Escribir
Ingrese los números que quiere ordenar de mayor a menor:
1
2
3
4
5
6
7
8
9
10
10.0 9.0 8.0 7.0 6.0 5.0 4.0 3.0 2.0 1.0
```

Números ingresados al azar

```
Ingrese los números que quiere ordenar de mayor a menor:
1
56
18
81
0
1
5198
1
3
6
5198.0 81.0 56.0 18.0 6.0 3.0 1.0 1.0 1.0 0.0
```

Descripción del problema 2:

Administrar un arreglo de 20 calificaciones mostrando promedio, máximo, mínimo y alumnos aprobados.

Diseño de la solución:

Requerimientos:

Se nos pide leer 20 calificaciones diferentes con las siguientes especificaciones:

- Mostrar el promedio, máximo, mínimo y alumnos que aprobaron
- No especifica de qué rango deben ser las calificaciones (negativas o positivas)
- No define qué tipo de valor van a ser las calificaciones (int, float o double)
- No especifica cómo se debe de hacer (qué estructuras de repetición se deben de usar)
- Cómo se deben leer los datos ingresados de los alumnos (terminal o Scanner)
- No se especifica si se debe de usar más de una clase o no

Solución:

Se decidió utilizar dos clases para realizar el ejercicio, una para los métodos y la otra para llamar a los métodos y analizar el arreglo. Se decidió instanciar una variable de la segunda clase para así poder acceder a sus métodos desde la clase principal (main) y en la segunda clase se utilizó un total de 5 métodos para realizar las diferentes acciones que se pide del programa, además de que en esta clase se utilizó varias variables double con el fin de poder utilizar un “finale”. Cabe mencionar que también se delimitó las calificaciones del 0 al 10.

Implementación:

Las partes más relevantes del código son los métodos de sacar máximo y mínimo, esto debe a que en ambos casos se utiliza un “for” para recorrer el arreglo y un “if” para poder comparar las posiciones del arreglo con la posición a la cual se iguala la variable “max/min”, para así evitar perder valores al momento de ejecutar el código. Esto ayudó a facilitar la estructura de los métodos y que sean más fáciles de comprender.

```

public void maximo(){
    max=calificaciones[0];
    for(int n=0;n<calificaciones.length;n++) {///  
        if(calificaciones[n]>max) {
            max=calificaciones[n];
        }
    }
}
public void minimo(){
    min=calificaciones[0];
    for(int n=0;n<calificaciones.length;n++) {///  
        if(calificaciones[n]<min) {
            min=calificaciones[n];
        }
    }
}
}

```

Observaciones:

En este ejercicio se tuvo dificultad en sacar el valor máximo y el mínimo debido a que al analizar la condición del “if” entre dos posiciones del arreglo ocurría que de repente se perdían los valores guardados al momento de ejecutarse el código lo que nos daba valores erróneos, por lo que al final se optó por cambiar la condición del “if” utilizando una posición del arreglo y la variable “max/min” que están apuntando a la posición inicial del arreglo.

Pruebas y resultados:

Números del 0 al 10

```

Ingrese las 20 calificaciones en un rango de 0 a 10
0
1
2
3
4
5
6
7
8
9
10
10
9
8
7
6
5
4
3
2
Las calificaciones son las siguientes: 0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 10.0, 9.0, 8.0, 7.0, 6.0, 5.0, 4.0, 3.0, 2.0,
El promedio es el siguiente: 5.45
La calificación máxima es de: 10.0
La calificación mínima es de: 0.0
Aprobaron un total de 10 alumnos

```

Números inválidos

```
Ingrese las 20 calificaciones en un rango de 0 a 10
```

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

```
6
```

```
7
```

```
8
```

```
9
```

```
10
```

```
11
```

```
Calificación inválida
```

```
Ingrese las 20 calificaciones en un rango de 0 a 10
```

```
-1
```

```
Calificación inválida
```

Al azar

```
~$ terminador -p754 (java Application) C:\Program Files\Java\jdk-23\bin\javaw.exe [20 Sep 2024, 11:41:20 PM] - 11.4.123 PMT: elapsed: 00:00:03.000 [pid: 2120]
```

```
Ingrese las 20 calificaciones en un rango de 0 a 10
```

```
2
```

```
5
```

```
6
```

```
8
```

```
7
```

```
4
```

```
1
```

```
0
```

```
3
```

```
9
```

```
6
```

```
5
```

```
4
```

```
0
```

```
1
```

```
2
```

```
5
```

```
8
```

```
9
```

```
7
```

```
Las calificaciones son las siguientes: 2.0, 5.0, 6.0, 8.0, 7.0, 4.0, 1.0, 0.0, 3.0, 9.0, 6.0, 5.0, 4.0, 0.0, 1.0, 2.0, 5.0, 8.0, 9.0, 7.0,
```

```
El promedio es el siguiente: 4.6
```

```
La calificación máxima es de: 9.0
```

```
La calificación mínima es de: 0.0
```

```
Aprobaron un total de 8 alumnos
```

Descripción del problema 3:

Simular un inventario con arreglos paralelos (nombre, precio y existencia), permitiendo búsquedas, actualización y cálculo del valor total del inventario.

Diseño de la solución:

Requerimientos:

El problema solicita administrar un inventario mediante arreglos paralelos con las siguientes especificaciones:

- Se debe almacenar el nombre de los productos.
- Se debe almacenar el precio de cada producto.
- Se debe almacenar la cantidad existente de cada producto.
- Los datos deben estar relacionados mediante la misma posición de los arreglos.
- Se debe permitir buscar un producto por su nombre.
- Se debe mostrar la información del producto encontrado.
- Se debe permitir actualizar la existencia de un producto.
- Se debe calcular el valor total del inventario.
- No se especifica la cantidad de productos que debe contener el inventario, por lo que se decidió utilizar 5 productos.
- No se especifica el tipo de dato para el precio y la existencia, por lo que se utilizaron los tipos primitivos `double` e `int`.
- No se especifica qué estructura de repetición se debe utilizar, por lo que se utilizaron ciclos `for`.
- No se especifica la forma de ingresar los datos, por lo que se utilizó la biblioteca `Scanner`.
- Se utilizaron estructuras de selección `if`, `else if` y `else` para realizar las búsquedas y validar las opciones.
- Se utilizó una clase propia para organizar la información y las operaciones relacionadas con el inventario.

Solución:

Se decidió utilizar tres arreglos paralelos para almacenar la información de los productos: un arreglo de tipo `String` para los nombres, uno de tipo `double` para los precios y uno de tipo `int` para las existencias.

Para relacionar los datos, cada producto ocupa la misma posición en los tres arreglos. Por ejemplo, el producto que se encuentra en la posición 0 de `nombres` tendrá su precio en la posición 0 de `precios` y su existencia en la posición 0 de `existencias`.

Se utilizó un ciclo `for` para recorrer los productos y realizar las búsquedas. Para las decisiones se utilizaron estructuras `if`, `else if` y `else`.

También se utilizó una clase llamada `Inventario`, la cual contiene los arreglos y los métodos necesarios para buscar productos, actualizar existencias y calcular el valor total del inventario.

Implementación

El programa se dividió en dos clases: `Inventario`, encargada de manejar los datos y operaciones del inventario, y `Main`, encargada de interactuar con el usuario mediante `Scanner`.

Implementación:

Observaciones:

La parte que presentó mayor dificultad fue relacionar correctamente los tres arreglos, ya que cada producto debe conservar la misma posición en los arreglos de nombre, precio y existencia. También fue necesario utilizar una variable de tipo **boolean** para determinar si el producto buscado se encontraba dentro del inventario.

Otra dificultad fue actualizar únicamente la existencia del producto seleccionado sin modificar los demás productos. Para solucionar esto se recorrió el arreglo de nombres y, cuando se encontró una coincidencia, se modificó la existencia utilizando el mismo índice.

El cálculo del valor total se realizó multiplicando el precio de cada producto por su existencia y posteriormente sumando todos los resultados.

Pruebas y resultados

Prueba 1: Mostrar inventario

Al seleccionar la opción **1**, el programa muestra todos los productos junto con su precio y existencia.

Pruebas y resultados:

Conclusiones:

Gonzaga Vargas Daniel Wilfrido

Conclusión Durante la realización de este ejercicio se reforzó el uso de arreglos, tipos de datos primitivos, clases, métodos y estructuras de selección en Java. Se desarrolló un programa capaz de simular un inventario mediante arreglos paralelos para almacenar el nombre de los productos, su precio y su existencia, además de permitir realizar búsquedas, actualizar cantidades y calcular el valor total del inventario. Durante la elaboración y ejecución del programa se presentaron algunos errores relacionados principalmente con la organización de los archivos y la ejecución de las clases desde la terminal. Entre ellos se encontraron problemas para localizar los archivos `.java`, errores al intentar ejecutar una clase que no contenía el método `main` y conflictos relacionados con el nombre del paquete y la clase principal. Estos errores permitieron comprender mejor la relación entre los archivos `.java`, los archivos `.class`, las clases y la forma correcta de compilar y ejecutar un programa desde la terminal. Finalmente, la práctica permitió comprender que no solamente es importante escribir correctamente el código, sino también organizar adecuadamente los archivos y conocer el proceso de

compilación y ejecución de un programa en Java. La resolución de los errores ayudó a identificar la función de la clase `Main` como punto de inicio del programa y a reforzar el manejo de herramientas como Visual Studio Code y la terminal.

Jimenez Arrieta Josue

En esta práctica aprendí a implementar arreglos paralelos para manipular y consultar información de distintos datos que se relacionan entre sí a través del mismo índice. Logré construir una lógica clara con estructuras básicas como while, switch y for, lo que me permitió crear un menú interactivo en consola para administrar el inventario, realizar búsquedas y actualizar precios y existencias sin complicaciones.

Jacques Sojo Leonardo:

En esta práctica aprendí a utilizar de manera eficiente los arreglos anidados además de comprender mejor la interacción entre las clases y sus métodos. También me

Referencias:

- Laboratorio Salas a y b, Microsoft Word- MADO-21_MPOO_v2 (Archivo PDF), <http://lcp02.fi-b.unam.mx/>.
-